Non-Determinism:

github.com/dataville/non_determinism_in_production

Charles Rice
Senior Data Scientist
March 2nd, 2026

linkedin.com/in/dataville
datainference.ai
github.com/dataville

When You Send the Same Question to an LLM Twice, You May Get Two Different Answers

You can’t assume identical behavior run to run.

This Is Acknowledged by the Companies That Build These Models

OpenAI

States the Chat Completions API is "non-deterministic by default", outputs will only be "mostly" identical, but not guaranteed.

Anthropic

Documentation states that results "will not be fully deterministic" — even at temperature zero.

Google Vertex AI

"Deterministic output isn't guaranteed" — model or parameter changes can cause variation.

What This Means for a Product Built on an LLM

The Scenario

Two users submit the same question. They may receive meaningfully different answers.

The same automated process run twice may produce different results.

Why This Matters

The same input can produce a correct answer on one run and an incorrect answer on another.
That difference is invisible unless you measure it.

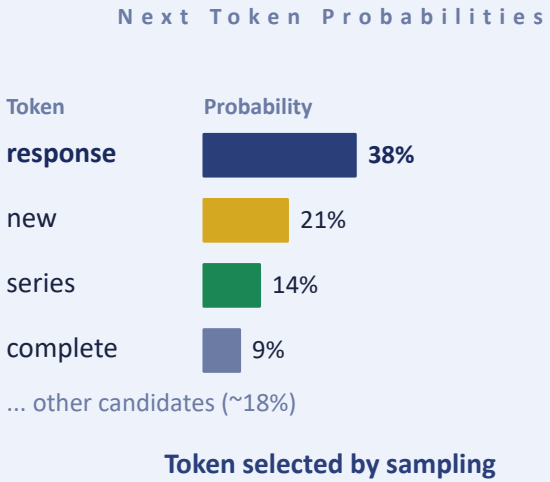
How LLMs Generate Text

An LLM Does Not Look Up Answers

LLMs Don’t Look Up Answers
They generate text one token at a time.

At each step the LLM:

- Computes probabilities for next tokens
- Samples from that distribution
- Higher probability ≠ guaranteed selection



Variability

Because each token is selected from a range of probable options rather than determined by a fixed rule, the same prompt can follow different paths through that selection process on each run.

SAME PROMPT

RUN 1

Response A

RUN 2

Response B

A small difference in one token early in the response can lead to a meaningfully different response overall.

Demo: Measuring Output Variance

What You Are About to See

Temperature controls how strongly the model prefers its top prediction when choosing the next word.
Lower values reduce randomness.
Higher values allow more variation.
Even at 0, identical outputs are not guaranteed.

The same prompt will be sent to a LLM five times under each of three conditions:
— Default temperature (no temperature argument passed to the API)
— Temperature 1.0 (a common default in many APIs)
— Temperature 0.0 (documented by providers as their most deterministic configuration)

Nothing else changes within each experiment except the temperature.

What the Results Show

Results across three experiments; same prompt, same model, 5 runs each:

Default temperature (unset)	**5 of 5 unique**
Temperature 1.0 (explicit)	**5 of 5 unique**
Temperature 0.0 (“designed to suppress variability”)	**3 of 5 unique**

The temperature 0.0 result is the one that matters.

Temperature = 0 is intended to minimize randomness.

It still produced different outputs across identical runs.

Providers document that temperature zero does not guarantee identical responses.

The question is not whether variation exists.
The question is whether correctness varies.

Can You Prompt Engineer Your Way Out of Non-Determinism?

Prompt engineering can reduce variability.
Clearer, more structured prompts narrow the range of likely outputs.

But output sampling remains probabilistic — even at temperature 0.

Prompt engineering changes the distribution the LLM samples from.
It does not remove the distribution.

Without measurement, you cannot tell whether a prompt change improved reliability or whether you're seeing normal run-to-run variation.

Failure Modes

Inconsistency When Two Users Get Different Answers

When the same question produces different outputs on different runs, users of the same product can receive conflicting information from the same system.

CASE STUDY: Moffatt v. Air Canada, 2024 BCCRT 149

Air Canada deployed a customer service chatbot that gave a passenger incorrect information about its refund policy. A tribunal ruled that Air Canada was responsible for everything its chatbot said, regardless of whether a human or an AI said it, and ordered the airline to pay compensation.

The chatbot did not malfunction. It generated a plausible-sounding answer that was wrong.

Moffatt v. Air Canada, 2024 BCCRT 149 · British Columbia Civil Resolution Tribunal, February 14, 2024 · https://www.canlii.org/en/bc/bccrt/doc/2024/2024bccrt149/2024bccrt149.html

When a Model Update Changes Your Product's Behavior

LLM providers update their models regularly. Those updates can change how your product behaves; without any change to your code or your prompts.

$$84\% \rightarrow 51\%$$

GPT-4 accuracy: prime vs. composite number identification
March 2023 → June 2023

Chen, Zaharia & Zou (2023)
Compared March vs. June 2023 model versions
Prime vs. composite task:
GPT-4 accuracy dropped 84% → 51%

The cause of the change was not publicly documented.

If you are not measuring your product's outputs systematically, you will not know this has happened.

Chen, Zaharia, Zou — "How is ChatGPT's Behavior Changing Over Time?" arXiv:2307.09009 (2023), Harvard Data Science Review · https://arxiv.org/abs/2307.09009

Drift: When Small Changes Have Large Consequences

LLM outputs are sensitive to small changes in input. Research published at ICLR 2024 systematically measured how minor formatting changes, differences in spacing, casing, and punctuation in prompts, produced large performance gaps across models and tasks.

Spacing

Differences in whitespace between tokens

Casing

Uppercase vs. lowercase in instructions

Punctuation

Presence or absence of commas, periods

The authors concluded that standard practice of evaluating models against a single prompt format substantially underestimates real-world variability. Routine updates, editing a prompt, adjusting wording, adding a new instruction, can have consequences that are difficult to predict without measurement.

Sclar et al., 2024 — "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design" ICLR 2024 · https://openreview.net/forum?id=RIu5lyNXjT

Demo: Implementing Evaluations with DeepEval

How the Demo Works

The demo has two LLMs running simultaneously:

THE PRODUCT

Generates answers to test questions.

→

THE JUDGE

- Scores each answer against written criteria.
- Uses a different model family.

The evaluation produces a score between 0 and 1 and a written reason for that score. If the score meets the threshold, the test passes. If it does not, it fails.

What Measurement Enables

Individual Scores Vary. Trends Don’t

So we measure:

- Single runs are noisy
- Reliability emerges across runs
- Track mean score and failure rate

If you evaluate once, you’re measuring noise. If you evaluate repeatedly, you’re measuring reliability.

Practical Starting Points

Before Any Tools: Observe Your Product

Take every prompt in your product. Run each one multiple times. Read the outputs carefully.

Ask yourself:

- How much do the outputs vary across runs?
- Are all of the variants correct, or do some fail?
- If this output feeds another part of the system, does a wrong answer here propagate silently through what comes after it?
- If this output reaches a user, what is the cost of a wrong answer?

Any prompt that produces inconsistent or incorrect outputs is a point of instability in your system. How much that matters depends on what the output does but you cannot make that assessment without first observing it.

You Cannot Evaluate Everything : So Prioritize

Exhaustive evaluation of every prompt on every run is not feasible in production. The cost is prohibitive and the latency impact is real.

The practical framework:

- Prompts that failed frequently in manual testing get evaluated most often
- Prompts that were stable get monitored at lower frequency
- No prompt is ever removed from the evaluation suite; a prompt that works today may not work after a model update, a data change, or a shift in how users interact with your system

Evaluation frequency is itself a decision that requires judgment. There is no universal formula. You are balancing coverage, cost, and risk, and that balance will shift as your system evolves.

Thank You

Questions

Charles Rice
Senior Data Scientist
linkedin.com/in/dataville
datainference.ai

Talk Resources

Code and slides:
github.com/dataville/non_determinism_in_production